

Unit 1: Assignment 2: Answer Key

B1. Write a program to check whether a given year is a leap year using nested if / logical operators.

Answer:

A year is a leap year if it is divisible by 4, but century years (divisible by 100) are leap years only if they are also divisible by 400.

```
#include <stdio.h>

int main() {
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if (year % 4 == 0) {
        if (year % 100 == 0) {
            if (year % 400 == 0)
                printf("%d is a leap year\n", year);
            else
                printf("%d is not a leap year\n", year);
        } else {
            printf("%d is a leap year\n", year);
        }
    } else {
        printf("%d is not a leap year\n", year);
    }

    return 0;
}
```

Equivalent single-condition version using logical operators:

```
if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
    printf("%d is a leap year\n", year);
else
    printf("%d is not a leap year\n", year);
```

B2. Write a program to find the largest among three numbers using if-else if-else.

Answer:

```
#include <stdio.h>

int main() {
    int a, b, c;

    printf("Enter three numbers: ");
```

```

scanf("%d %d %d", &a, &b, &c);

if (a >= b && a >= c)
    printf("%d is the largest\n", a);
else if (b >= a && b >= c)
    printf("%d is the largest\n", b);
else
    printf("%d is the largest\n", c);

return 0;
}

```

B3. Write a program that reads a character and determines whether it is a vowel or a consonant.

Answer:

```

#include <stdio.h>

int main() {
    char ch;

    printf("Enter a character: ");
    scanf("%c", &ch);

    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
        ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
        printf("%c is a vowel\n", ch);
    } else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
        printf("%c is a consonant\n", ch);
    } else {
        printf("%c is not an alphabet\n", ch);
    }

    return 0;
}

```

B4. Write a program to calculate the grade of a student based on marks entered (using else if ladder): $\geq 90 \rightarrow A$, $\geq 75 \rightarrow B$, $\geq 60 \rightarrow C$, $< 60 \rightarrow \text{Fail}$

Answer:

```

#include <stdio.h>

int main() {
    int marks;

    printf("Enter marks: ");
    scanf("%d", &marks);

    if (marks >= 90)

```

```

    printf("Grade: A\n");
else if (marks >= 75)
    printf("Grade: B\n");
else if (marks >= 60)
    printf("Grade: C\n");
else
    printf("Fail\n");

return 0;
}

```

Note: the conditions must be checked from highest to lowest (≥ 90 first), because else if only reaches a later condition when all earlier ones are false.

B5. Predict the output. Explain why the output is not 2.5, and how to fix it.

```

int a = 5, b = 2;
float c;
c = a / b;
printf("%f", c);

```

Answer:

Output: 2.000000

Explanation: a and b are both int, so a / b performs integer division, which discards (truncates) the fractional part before the result is ever converted to float. 5 / 2 evaluates to the integer 2 first; only after that is 2 converted to 2.0 and stored in c. Declaring c as float does not change how a / b itself is evaluated — the division happens using the types of a and b, not the type of the variable it is later assigned to.

Fix: convert (cast) at least one operand to float before the division takes place, so floating-point division is used instead of integer division:

```

c = (float)a / b;
// or
c = a / (float)b;

```

Either version now correctly gives c = 2.500000.

B6. Write a program to check whether a triangle is valid given its three angles (sum must equal 180, and each angle > 0).

Answer:

```

#include <stdio.h>

int main() {
    int angle1, angle2, angle3;

    printf("Enter three angles of the triangle: ");
    scanf("%d %d %d", &angle1, &angle2, &angle3);

    if (angle1 > 0 && angle2 > 0 && angle3 > 0 &&

```

```

    (angle1 + angle2 + angle3 == 180)) {
        printf("Valid triangle\n");
    } else {
        printf("Invalid triangle\n");
    }

    return 0;
}

```

B7. Explain operator precedence with an example involving +, *, and (). Evaluate: `int result = 10 + 2 * 3 - (4 / 2);`

Answer:

Operator precedence determines the order in which operators are applied in an expression containing more than one operator. In C, parentheses () have the highest precedence and are evaluated first (innermost first); multiplication * and division / have higher precedence than addition + and subtraction -; and among operators of equal precedence, evaluation generally proceeds left to right (associativity).

Step-by-step evaluation of `10 + 2 * 3 - (4 / 2)`:

- Parentheses first: $(4 / 2) = 2$
- Multiplication next: $2 * 3 = 6$
- Expression becomes: $10 + 6 - 2$
- Addition and subtraction, left to right: $10 + 6 = 16$, then $16 - 2 = 14$

result = 14

B8. Write a program to input a number and check if it is divisible by both 3 and 5 using logical && operator.

Answer:

```

#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 3 == 0 && num % 5 == 0)
        printf("%d is divisible by both 3 and 5\n", num);
    else
        printf("%d is not divisible by both 3 and 5\n", num);

    return 0;
}

```

B9. What is implicit type conversion? Predict the output.

```
char ch = 'A';
int x = ch + 1;
printf("%d %c", x, x);
```

Answer:

Implicit type conversion (also called automatic type conversion or type promotion) happens when the compiler automatically converts one data type to another without the programmer explicitly writing a cast, in order to carry out an operation or assignment. It commonly happens when "smaller" types (like char or short) are used in arithmetic expressions — they are automatically promoted to int, and when mixing int and float, the int is promoted to float.

In this example, ch holds the character 'A', whose ASCII value is 65. When ch + 1 is evaluated, ch is implicitly converted (promoted) to an int (65), and 1 is added, giving 66. This value 66 is then stored in the int variable x.

Output: 66 B

%d prints x as the integer 66. %c prints x reinterpreted as a character — ASCII code 66 corresponds to the character 'B'.

B10. Write a program using nested if-else to categorize a person's BMI (Underweight/Normal/Overweight) based on input weight and height.

Answer:

BMI is calculated as weight (kg) divided by height (m) squared. A common classification: BMI < 18.5 → Underweight, 18.5 ≤ BMI < 25 → Normal, BMI ≥ 25 → Overweight.

```
#include <stdio.h>

int main() {
    float weight, height, bmi;

    printf("Enter weight in kg: ");
    scanf("%f", &weight);
    printf("Enter height in meters: ");
    scanf("%f", &height);

    bmi = weight / (height * height);
    printf("BMI = %.2f\n", bmi);

    if (bmi < 18.5) {
        printf("Category: Underweight\n");
    } else {
        if (bmi < 25.0)
            printf("Category: Normal\n");
        else
            printf("Category: Overweight\n");
    }

    return 0;
}
```

```
}
```

B11. Differentiate between if-else and else if ladder with a program example where else if is more efficient.

Answer:

- if-else: a single two-way decision — exactly one condition is tested, and either the if block or the else block runs. Multiple independent if-else statements (not chained) each test their own condition separately, even after an earlier one has already been satisfied.
- else if ladder: a chain of conditions tested in sequence, where each condition is only checked if all the previous ones were false. As soon as one condition is true, its block runs and the rest of the ladder is skipped entirely.

Example — grading a student, using separate if statements (inefficient: every condition is always checked, even after a match is already found):

```
if (marks >= 90) printf("A\n");
if (marks >= 75 && marks < 90) printf("B\n");
if (marks >= 60 && marks < 75) printf("C\n");
if (marks < 60) printf("Fail\n");
```

Same logic using an else if ladder (efficient: once a condition matches, the remaining checks are skipped, and each condition can be written simply without repeating the lower bound):

```
if (marks >= 90) printf("A\n");
else if (marks >= 75) printf("B\n");
else if (marks >= 60) printf("C\n");
else printf("Fail\n");
```

The else if ladder is more efficient here because it stops evaluating further conditions as soon as a match is found, and it avoids the extra compound conditions (&&) needed to make independent if statements mutually exclusive.

B12. Predict the output and explain the operator behavior.

```
int a = 5;
printf("%d %d %d", a++, ++a, a);
```

Answer:

This expression is technically undefined behavior in C, and students should be careful not to write code like this in practice. The reason: the C standard does not fix the order in which function arguments are evaluated, and `a` is modified more than once (by `a++` and by `++a`) without a "sequence point" between those modifications — the standard does not guarantee what value each use of `a` will see.

What each operator does individually:

- `a++` (post-increment) — uses the current value of `a` in the expression, then increments `a` afterward.
- `++a` (pre-increment) — increments `a` first, then uses the new value in the expression.

Because the overall behavior is undefined, different compilers (or even different optimization settings) are allowed to produce different results. On common compilers such as GCC, this

program was tested and produced the output 6 7 7, but this exact result should not be relied upon or assumed for other compilers.

The key lesson for the exam: never modify the same variable more than once within a single expression without a sequence point (such as the end of a full statement). Write it as separate statements instead, e.g.:

```
int a = 5;
int p = a++; // p = 5, a becomes 6
int q = ++a; // a becomes 7, q = 7
printf("%d %d %d", p, q, a); // well-defined: 5 7 7
```

B13. Write a program using nested if that first checks whether a number lies between 1 and 100, and if so, further classifies it as belonging to the range 1–50 or 51–100.

Answer:

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num >= 1 && num <= 100) {
        if (num <= 50)
            printf("%d lies in the range 1-50\n", num);
        else
            printf("%d lies in the range 51-100\n", num);
    } else {
        printf("%d is not between 1 and 100\n", num);
    }

    return 0;
}
```

B14. Write a menu-driven program using switch to convert temperature between Celsius, Fahrenheit, and Kelvin, based on the user's choice of conversion (e.g., 1 = C to F, 2 = F to C, 3 = C to K).

Answer:

```
#include <stdio.h>

int main() {
    int choice;
    float temp, result;

    printf("1. Celsius to Fahrenheit\n");
    printf("2. Fahrenheit to Celsius\n");
```

```

printf("3. Celsius to Kelvin\n");
printf("Enter your choice: ");
scanf("%d", &choice);

printf("Enter the temperature: ");
scanf("%f", &temp);

switch (choice) {
    case 1:
        result = (temp * 9.0 / 5.0) + 32;
        printf("Result = %.2f F\n", result);
        break;
    case 2:
        result = (temp - 32) * 5.0 / 9.0;
        printf("Result = %.2f C\n", result);
        break;
    case 3:
        result = temp + 273.15;
        printf("Result = %.2f K\n", result);
        break;
    default:
        printf("Invalid choice\n");
}

return 0;
}

```

B15. Write a program using switch-case to print the number of days in a given month (1–12). For month 2 (February), use a nested if inside the switch case to additionally check for a leap year.

Answer:

```

#include <stdio.h>

int main() {
    int month, year, days;

    printf("Enter month (1-12): ");
    scanf("%d", &month);

    switch (month) {
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:
            days = 31;
            printf("Days = %d\n", days);
            break;
        case 4: case 6: case 9: case 11:

```

```

    days = 30;
    printf("Days = %d\n", days);
    break;
case 2:
    printf("Enter year: ");
    scanf("%d", &year);
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        days = 29;
    else
        days = 28;
    printf("Days = %d\n", days);
    break;
default:
    printf("Invalid month\n");
}

return 0;
}

```

B16. Compare nested if-else and switch-case statements. Give one example situation where switch is clearly the better choice, and one where it cannot be used at all even though if-else can.

Answer:

Comparison:

- if-else can test any kind of condition — relational (>, <, >=), logical (&&, ||), floating-point comparisons, ranges, function calls that return true/false, etc. It works with any expression that evaluates to true or false.
- switch-case can only test a single variable/expression of integer or character type for exact equality against constant values (each case label must be a constant expression). It cannot test ranges directly, floating-point values, logical combinations of variables, or general boolean conditions.
- When there are many discrete, exact-match options, switch is usually more readable and can be more efficient than a long if-else if chain.

Example where switch is clearly the better choice — printing a day name from a number 1–7 (a single integer variable being matched against several exact discrete values):

```

switch (day) {
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    // ... and so on
}

```

Example where switch cannot be used at all, even though if-else can — checking whether a person's height (a float) lies within a range, or a condition combining two different variables:

```

if (height >= 1.5 && height <= 1.8)
    printf("Height is within normal range\n");

```

This cannot be written with switch-case because the condition is a floating-point range check involving a relational/logical expression, not an equality test of a single integer/char variable against constant values — exactly the hint given in the question.

B17. Write a program using nested if to determine ticket pricing at a cinema: first check if the person is a student (discount applies), then within that, check if the show is a weekday or weekend (different discount percentages for each combination).

Answer:

Assumed base ticket price and discount rules (these can be adjusted to match any specific values given in class): base price = 200; students get 30% off on weekdays and 15% off on weekends; non-students get 10% off on weekdays and no discount on weekends.

```
#include <stdio.h>

int main() {
    char isStudent, dayType; // 'y'/'n' and 'w' (weekday) / 'e' (weekend)
    float basePrice = 200.0, finalPrice;

    printf("Are you a student? (y/n): ");
    scanf(" %c", &isStudent);
    printf("Is the show on a weekday or weekend? (w/e): ");
    scanf(" %c", &dayType);

    if (isStudent == 'y' || isStudent == 'Y') {
        if (dayType == 'w' || dayType == 'W')
            finalPrice = basePrice - (basePrice * 0.30); // 30% off
        else
            finalPrice = basePrice - (basePrice * 0.15); // 15% off
    } else {
        if (dayType == 'w' || dayType == 'W')
            finalPrice = basePrice - (basePrice * 0.10); // 10% off
        else
            finalPrice = basePrice; // no discount
    }

    printf("Ticket price = %.2f\n", finalPrice);

    return 0;
}
```